FIFA 19 SCOUT PLAYER

Laporan Proyek & Penjelasan Kode

Mata Kuliah: Intro to Computer Programming — Final Exam (Take-Home)

Nama : _____

NIM : _____

Tanggal: November 2026

1. Ringkasan Program

FIFA 19 Scout Player adalah program CLI (Command Line Interface) berbasis Python yang membantu scout sepak bola mencari **pemain muda berbakat** dari dataset Kaggle FIFA 19. Program hanya menggunakan **standard library** (tanpa pandas/numpy), dan semua pengurutan ditulis dengan **Selection Sort manual** tanpa `sorted()` / `.sort()`.

Saat dimulai, program memuat CSV dan **mencetak jumlah record yang berhasil dimuat serta jumlah baris rusak yang dilewati** (sesuai ketentuan soal):

```
=== FIFA 19 SCOUT PLAYER ===
Data Berhasil Dimuat: 1983 pemain | Baris Dilewati: 5
```

Program punya 3 fitur utama plus pengelolaan slot komparasi:

Menu	Fitur	Modul
1	**Cari Berdasarkan Nama** — pencarian tanpa peduli huruf besar/kecil dan aksen (ketik `mbappe` tetap menemukan "K. Mappé")	`cari_nama.py`
2	**Filter Pemain** — umur, overall, potensi, posisi, dan budget gaji disaring *sekaligus*; ketik **0 atau Enter** untuk melewatkan kriteria	`filter_pemain.py`
3	**Komparasi Pemain Terpilih** — semua pemain di slot komparasi ditampilkan berdampingan dalam satu tabel	`komparasi.py`
4	**Hapus Pemain dari Slot Komparasi** — mengeluarkan satu pemain yang tidak jadi dibandingkan	`komparasi.py`
0	Keluar	`main.py`

Cara Menjalankan

```
# dari folder root proyek (PowerShell)
python main.py              # jalankan program
python test_project.py      # jalankan 19 unit test
python -m unittest          # cara kedua, hasil sama
python trim_dataset.py      # (sekali saja) potong dataset asli
```

2. Dataset & Penyiapan

- **Sumber:** FIFA 19 Complete Player Dataset (javagarm), Kaggle — 18.207 baris, 89 kolom, encoding cp1252.
- **Alasan dipilih:** menggabungkan identitas, atribut, dan data finansial dalam satu file — cocok untuk filter scouting multi-kriteria.
- **Trim** (`trim_dataset.py`): disaring menjadi pemain **umur ≤ 21** dan **Potential ≥ 75**; baris tanpa `Club` dikeluarkan (penanganan sel kosong); hasilnya 1.988 baris disimpan ke `fifa19_young.csv`

(UTF-8, sefolder dengan `main.py`) dengan 13 kolom.

- **Loading** (`data.py`): setiap baris divalidasi `is_valid_row()`; baris rusak dilewati tanpa crash dan **dilaporkan jumlahnya** — 1.983 pemain dimuat, 5 baris dilewati.
- **Kolom dengan sel kosong:** `Release Clause` kosong pada 270 baris yang tetap dimuat; pemain seperti itu ditampilkan dengan nilai default `€0`. Kolom kosong lainnya (Position, Vision, Reactions, Preferred Foot) hanya ada di 5 baris rusak yang memang di-skip loader.

Kelompok	13 Kolom yang Dipakai
Identitas	`Name, Age, Club, Position, Preferred Foot`
Rating	`Overall, Potential`
Finansial	`Value, Wage, Release Clause`
Atribut inti	`Vision, Reactions, Composure`

3. Struktur Proyek

```
Scout Player/
├── main.py                 # Entry point: menu utama CLI
├── fifa19_young.csv        # Dataset hasil trim (UTF-8, 13 kolom)
├── test_project.py         # Runner seluruh unit test
├── trim_dataset.py         # Script pemotong dataset (sekali jalan)
├── data/
│   └── fifa19.csv          # Dataset asli Kaggle (cp1252, sumber trim)
└── pythonapp/
    ├── clean_text.py       # Util: normalisasi teks, parse/format uang, input angka
    ├── data.py             # Class Player & ScoutDatabase, loader CSV + validasi
    ├── cari_nama.py        # Fitur 1 + tabel hasil + profil + pemilihan pemain
    ├── filter_pemain.py    # Fitur 2: filter gabungan + Selection Sort manual
    ├── komparasi.py        # Fitur 3 & 4: komparasi multi-pemain + hapus dari slot
    └── tests/              # 3 file unit test (19 test, unittest)
```

4. Penjelasan Kode per Modul

4.1 clean_text.py — utilitas teks, uang, dan input

Konsep: Variables & Constants, Strings, Conditional Statements, Loops, Functions

Konstanta huruf kapital (Core Concept: Variables & Constants) dan tabel aksen untuk normalisasi nama pemain:

```
CURRENCY_SYMBOL = "€"
MILLION_MULTIPLIER = 1_000_000.0
THOUSAND_MULTIPLIER = 1_000.0

# Tabel penggantian karakter beraksen (contoh: é -> e)
ACCENT_TABLE = str.maketrans(
    "éèêëáàâãäíìîïóòôõöúùûüñçćšžø",
    "eeeeeaaaaaiiiiooooouuuuncszo"
)
```

clean_text() merantai metode string (Core Concept: String Manipulation): `.strip()` buang spasi ujung, `.lower()` agar pencarian tidak peduli huruf besar/kecil, `.translate()` hapus aksen, `.replace()` ubah tanda hubung, `.split()+join()` rapikan spasi ganda. Ini alasan ketik `joao felix` tetap menemukan "João Félix":

```
def clean_text(text: str) -> str:
    if not isinstance(text, str):
        return ""
    cleaned = text.strip().lower().translate(ACCENT_TABLE).replace("-", " ")
    return " ".join(cleaned.split())
```

parse_currency() mengubah teks finansial menjadi angka dengan *percabangan minimal 3 kondisi* (Core Concept: Conditional Statements) dan `try/except` agar input rusak mengembalikan 0.0, bukan error:

```
try:
    if raw.endswith("M"):                    # '€110.5M' -> 110_500_000.0
        return float(raw[:-1]) * MILLION_MULTIPLIER
    elif raw.endswith("K"):                  # '€565K'   -> 565_000.0
        return float(raw[:-1]) * THOUSAND_MULTIPLIER
    else:
        return float(raw)
except ValueError:
    return 0.0
```

ask_int() membaca input angka dengan loop validasi (Core Concept: Loops) — input salah diminta ulang, program tidak pernah crash:

```
def ask_int(prompt: str, min_val: int, max_val: int) -> int:
    while True:
        try:
            val = int(input(prompt).strip())
            if min_val <= val <= max_val:
                return val
            print(f"  [!] Masukkan angka antara {min_val} dan {max_val}.")
        except ValueError:
            print("  [!] Input harus berupa angka bulat.")
```

4.2 data.py — class Player, ScoutDatabase, dan loader CSV

Konsep: OOP (bonus), Data Types, Operators, Lists & Dictionaries, File I/O

Player adalah *Record Class* (Bonus OOP +5): satu objek = satu pemain. Konstruktor mengonversi **tipe data** dari teks CSV (Core Concept: Data Types) — `str → int` untuk rating, `str → float` lewat `parse_currency` untuk uang:

```
self.age: int = int(data.get("Age", 0))                # '19' -> 19
self.overall: int = int(data.get("Overall", 0))
self.value_num: float = parse_currency(self.value_str) # '€81M' -> 81000000.0
```

Operator aritmatika menghitung dua nilai turunan (Core Concept: Operators — soal mensyaratkan minimal dua): *growth* dengan pengurangan, dan *gaji tahunan* dengan perkalian:

```
# Nilai Turunan 1: Growth dihitung dengan operator pengurangan '-'
self.growth: int = self.potential - self.overall        # 95 - 88 = +7

# Nilai Turunan 2: Gaji tahunan dihitung dengan operator perkalian '*' (52 minggu)
self.yearly_wage: float = self.wage_num * 52            # €100K -> €5.2M/tahun
```

Dictionary & loop menyimpan atribut (Core Concept: Lists & Dictionaries). `try/except` mengisi 0 jika data kosong:

```
self.stats: dict[str, int] = {}
for key in STAT_KEYS:                    # ["Vision", "Reactions", "Composure"]
    try:
        self.stats[key] = int(data.get(key, 0))
    except (ValueError, TypeError):
        self.stats[key] = 0
```

Pengelompokan posisi memakai **dictionary bersarang + loop + break** — kode posisi (RM) dipetakan ke grup (MID):

```
POSITION_GROUPS = {
    "FWD": ["ST", "CF", "LW", "RW"],
    "MID": ["CAM", "CM", "CDM", "LM", "RM"],
    "DEF": ["CB", "LB", "RB", "LWB", "RWB"],
    "GK": ["GK"]
}
self.position_group: str = "OTHER"
for group, positions in POSITION_GROUPS.items():
    if self.position.upper() in positions:
        self.position_group = group
        break
```

ScoutDatabase adalah *Collection Class* (Bonus OOP +5) — menampung semua objek Player dalam satu list. Method `add_player` memvalidasi tipe dengan `isinstance`, dan *dunder method* `__len__` / `__getitem__` membuat objek bisa dipakai seperti list: `len(db)` dan `db[0]`:

```
class ScoutDatabase:
    def __init__(self):
        self.players: list[Player] = []

    def add_player(self, player: Player) -> None:
        if isinstance(player, Player):
            self.players.append(player)

    def __len__(self) -> int:
        return len(self.players)

    def __getitem__(self, index):
        return self.players[index]
```

load_dataset() membaca CSV dengan `csv.DictReader` (Core Concept: File I/O). Setiap baris divalidasi `is_valid_row()`; baris rusak *dilewati dan dihitung*, bukan membuat program error — jumlahnya dilaporkan saat program mulai:

```
with open(filepath, mode="r", encoding=DEFAULT_ENCODING, errors="replace") as file:
    for row in csv.DictReader(file):          # tiap baris = dictionary
        if is_valid_row(row):
            database.add_player(Player(row))
        else:
            skipped_count += 1
return database, skipped_count                # (ScoutDatabase, jumlah baris dilewati)
```

4.3 cari_nama.py — Fitur 1: Cari Berdasarkan Nama

Konsep: Functions, Lists (list comprehension + slicing), Strings

search_by_name() memakai *list comprehension* — nama user dan nama pemain dua-duanya dibersihkan dulu dengan `clean_text` sehingga pencarian fleksibel:

```
def search_by_name(players: list[Player], query: str) -> list[Player]:
    q = clean_text(query)
    if not q:
        return []
    return [p for p in players if q in clean_text(p.name)]
```

print_table() memakai **f-string dengan format alignment** (`:<20` = rata kiri lebar 20) supaya kolom tabel sejajar rapi di terminal — tanpa library tabel — serta **slicing** `players[:limit]` untuk mengambil N hasil pertama:

```
sub = players[:limit]                          # slicing: N item pertama
print(f"{'No':<3} | {'Nama':<20} | {'Usia':<4} | {'Klub':<18} | {'Pos':<4} | ...")
for i, p in enumerate(sub, start=1):
    print(f"{i:<3} | {p.name[:20]:<20} | {p.age:<4} | {p.club[:18]:<18} | ...")
```

print_detail_card() menampilkan profil lengkap, termasuk kedua nilai turunan (pertumbuhan dan gaji tahunan):

```
print(f"  Potensi        : {p.potential} (Pertumbuhan: +{p.growth})")
print(f"  Gaji           : {p.wage_str}/minggu (setahun: {format_currency(p.yearly_wage)})")
```

handle_selected_player() — submenu setelah pemain dipilih. Mengecek *duplicate* dengan `any()` sebelum menambah ke slot komparasi:

```
if any(p.name == player.name for p in slot_compare):
    print(f"  [!] {player.name} sudah ada di dalam slot komparasi.")
else:
    slot_compare.append(player)
    print(f"  [OK] '{player.name}' berhasil di-select! (total {len(slot_compare)} pemain)")
```

4.4 filter_pemain.py — Fitur 2: Filter Gabungan + Sorting Manual

Konsep: Loops (nested), Operators (perbandingan + logika), Functions (parameter default)

manual_sort() — Selection Sort yang ditulis sendiri dengan **loop bersarang** (Core Concept: Loops): loop luar menandai posisi yang mau diisi, loop dalam mencari nilai terbesar/terkecil di sisanya, lalu ditukar. List aslinya tidak berubah karena disalin dulu dengan `list(items)`:

```
def manual_sort(items: list, key_func, descending: bool = True) -> list:
    result = list(items)
    n = len(result)
    for i in range(n):
        target_idx = i
        for j in range(i + 1, n):              # cari kandidat terbaik di sisa list
            if key_func(result[j]) > key_func(result[target_idx]):
                target_idx = j
        if target_idx != i:
            result[i], result[target_idx] = result[target_idx], result[i]   # tukar
    return result
```

filter_combined() menyaring **semua kriteria sekaligus** dengan operator perbandingan (`<`, `>`) dan operator logika `and` (Core Concept: Operators). Setiap kriteria bersifat opsional lewat **parameter default** `None` (Core Concept: Functions) — kalau `None`, kriteria itu dilewati:

```
def filter_combined(players, min_age=None, max_age=None, min_overall=None,
                    max_potential=None, position_query=None, max_wage=None):
    results = []
    for p in players:
        if min_age is not None and p.age < min_age:       # operator logika 'and'
            continue                                      # 'continue' = lewati pemain ini
        if max_age is not None and p.age > max_age:
            continue
        ...
        results.append(p)
    return results
```

menu_filter_combined() menggabungkan semuanya: input dibaca dengan `ask_optional_int`, posisi dan budget divalidasi lewat `ask_position()` / `ask_budget()` (input tidak dikenal diminta ulang, tidak pernah bocor ke filter), hasil difilter lalu **diurutkan** dengan `manual_sort` berdasarkan potensi tertinggi:

```
return manual_sort(results, key_func=lambda p: (p.potential, p.overall), descending=True)
```

4.5 komparasi.py — Fitur 3 & 4: Komparasi Multi-Pemain + Hapus dari Slot

Konsep: Lists (list of lists), Strings (join), f-string dinamis

Tabel komparasi dibangun **dinamis mengikuti jumlah pemain di slot**: header dibuat dengan `join` + generator, dan setiap baris statistik disimpan sebagai list berisi nilai semua pemain:

```
col_w = 16
header = f"{'Statistik':<16} | " + " | ".join(f"{p.name[:col_w]:<{col_w}}" for p in slot_compare)

rows = [
    ("Usia",     [str(p.age) for p in slot_compare]),
    ("Overall",  [str(p.overall) for p in slot_compare]),
    ...
]
for label, vals in rows:
    print(f"{label:<16} | " + " | ".join(f"{str(v)[:col_w]:<{col_w}}" for v in vals))
```

remove_from_slot() — mengeluarkan satu pemain dari slot komparasi (menu 4). Daftar pemain ditampilkan bernomor, lalu pemain dipilih dengan `ask_int` dan dihapus dari list dengan `pop()`:

```
for i, p in enumerate(slot_compare, start=1):
    print(f"  {i}. {p.name} ({p.club} | Pos: {p.position} | OVR: {p.overall} | POT:
{p.potential})")

idx = ask_int(f"Pilih nomor pemain yang ingin dihapus (1-{len(slot_compare)}, 0=batal): ", 0,
len(slot_compare))
if idx > 0:
    removed = slot_compare.pop(idx - 1)
    print(f"  [OK] '{removed.name}' dihapus dari slot komparasi. Sisa: {len(slot_compare)}
pemain.")
```

4.6 main.py — Entry Point & Menu Utama

Konsep: Loops (while + break), Modularitas (import)

`main.py` sengaja dibuat tipis (hanya import + menu) — seluruh logika ada di modul `pythonapp/` yang ditambahkan ke `sys.path`. Saat mulai, program mencetak jumlah record yang dimuat dan baris yang dilewati. Menu utama berjalan dengan **loop** `while True` yang berhenti lewat `break` saat user memilih 0:

```
db, skipped = load_dataset(DEFAULT_CSV_FILE)
print(f"Data Berhasil Dimuat: {len(db)} pemain | Baris Dilewati: {skipped}")
slot_compare = []

while True:
    print("\nMENU UTAMA:")
    print(f"  Slot Komparasi ({len(slot_compare)} pemain)")
    ...
    opt = ask_int("Pilih menu (0-4): ", 0, 4)

    if opt == 1:
        results = search_by_name(db.get_all(), q)
        select_from_results(results, slot_compare)
    elif opt == 2:
        run_filter_feature(db, slot_compare)
    elif opt == 3:
        run_comparison_feature(slot_compare)
    elif opt == 4:
        remove_from_slot(slot_compare)
    elif opt == 0:
        print("\nTerima kasih telah menggunakan Scout Player. Sampai jumpa!")
        break
```

5. Tabel 8 Konsep Inti

#	Konsep	File	Fungsi / Lokasi
1	Variables & Constants	`clean_text.py`, `data.py`	`CURRENCY_SYMBOL`, `MILLION_MULTIPLIER`, `DEFAULT_CSV_FILE`, `REQUIRED_FIELDS`, `POSITION_GROUPS`
2	Data Types	`data.py`	`Player.__init__` — konversi `str→int` dan `str→float`
3	Operators	`data.py`, `filter_pemain.py`	Dua nilai turunan: `growth = potential - overall`, `yearly_wage = wage_num * 52`; perbandingan + `and` di `filter_combined`
4	Conditional Statements	`clean_text.py`, `data.py`	`parse_currency` (if/elif/else K/M), `is_valid_row`
5	Loops	`filter_pemain.py`, `main.py`, `data.py`	`manual_sort` (nested for), menu `while True`, `for` pembaca CSV
6	Functions	semua modul	parameter opsional default, mis. `filter_combined(..., min_age=None)`
7	Lists & Dictionaries	`data.py`, `cari_nama.py`	`stats` (dict), `POSITION_GROUPS` (dict), `slot_compare` (list), slicing `players[:limit]`, baris `DictReader`
8	String Manipulation	`clean_text.py`	`clean_text` (strip, lower, translate, replace, split, join)

Tambahan: **Input/Output** dengan `input()` + validasi loop + f-string kolom sejajar ada di `ask_int` dan **File I/O** di `data.py` / `trim_dataset.py`.

6. Unit Test

19 test berjalan otomatis dengan **unittest** — bisa lewat `python test_project.py` maupun `python -m unittest` dari folder root. Hasil: **OK (19 tests)**.

File Test	Yang Diuji
`test_clean_text.py`	`clean_text` (aksen, spasi, bad input), `parse_currency` (M/K/plain, input rusak), `format_currency` (M/K/kecil/nol/negatif)
`test_data.py`	`Player` (tipe atribut, growth, pengelompokan posisi, `__str__`), `ScoutDatabase` (add/len/indexing/validasi), `is_valid_row`, `load_dataset` (1.983 dimuat / 5 dilewati, file hilang tidak crash)
`test_fitur.py`	`search_by_name`, `manual_sort` (naik & turun), `filter_combined` (semua kriteria + kasus kosong)

7. Keterbatasan

- Dataset dipotong hanya ke pemain muda (umur ≤ 21, Potential ≥ 75) — pemain senior tidak bisa dicari.
- Hanya 13 kolom yang tersimpan; atribut teknis lain (Finishing, Dribbling, dll.) tidak tersedia.
- `Release Clause` kosong pada 270 baris dimuat sebagai `€0` (nilai default).
- Batas input filter diambil dari rentang data aktual (umur 15–21, OVR 40–90, POT 75–95, gaji €1K–€155K); angka di luar rentang ditolak dan diminta ulang.
- Tabel komparasi memotong nama pemain 16 karakter agar muat di lebar terminal.

8. Video Penjelasan

Link video: *(ditambahkan setelah perekaman)*